

Mini-Exposé (v3): Kontexttransparenz für Coding Agents

Arbeitstitel: Kontexttransparenz für Coding Agents: Inspizierbares, graphbasiertes Kontext-Engineering und seine Wirkung auf Effizienz und Entwicklerkontrolle

Bezug zur Ausschreibung: „Mehr Effizienz mit Coding Agents? Kontexttransparenz und Kontext-Engineering“

1 Problemstellung

Coding Agents (Claude Code, Cursor, SWE-agent) planen Aufgaben, lesen Dateien und koordinieren Änderungen über mehrere Module hinweg. Ihr Erfolg hängt entscheidend von der **Kontextbereitstellung** ab: Was der Agent über die Codebase weiß, bestimmt, ob er korrekt löst oder halluziniert.

Dabei bestehen zwei offene Probleme:

Ineffizientes Kontext-Engineering. Agents explorieren Codebases reaktiv und dabei nachweislich verschwenderisch: Sie rufen systematisch mehr Kontext ab als sie nutzen und besuchen Dateien wiederholt (ContextBench, Li et al. 2026). Statische Kontext-Dateien (CLAUDE.md, AGENTS.md) helfen nur begrenzt: handgeschriebene verbessern die Lösungsrate um ~4 %, LLM-generierte verschlechtern sie sogar, beide erhöhen die Inferenzkosten um > 20 % (Gloaguen et al. 2026).

Intransparente Kontextauswahl. Produziert ein Agent eine falsche Lösung, kann der Entwickler nicht nachvollziehen, *welcher Kontext zu welcher Entscheidung* geführt hat. Kontext ist eine Black Box. Bestehende Transparenz-Werkzeuge setzen auf Trajektorien-Ebene an: AgentStepper (2026) ermöglicht schrittweises Debugging laufender Agent-Trajektorien (Breakpoints, Live-Editing von Prompts und Tool-Aufrufen), adressiert damit aber Fehler in der Agent-Implementierung, nicht die Bewertung der Kontextauswahl **vor** dem Start. Ob Entwickler die Kontextauswahl vor der Agentenausführung bewerten und korrigieren können und ob das die Ergebnisqualität verbessert, ist bislang kaum systematisch untersucht. Der Bedarf ist dabei dokumentiert: Eine Beobachtungsstudie mit 19 Entwicklern identifiziert fehlendes projektspezifisches Wissen des Agenten als Haupthindernis im Praxiseinsatz, und eine Interviewstudie mit 17 Entwicklern (Dhanorkar et al. 2026) zeigt, dass Entwickler präventive Kontrolle über Agents („a priori control“, Co-Planning) bereits zu praktizieren versuchen, ihnen dafür

aber Werkzeuge fehlen; genau hier braucht es einen kontrollierbaren Eingriffspunkt.

2 Forschungslücke und Abgrenzung

Graphbasierte Kontextzusammenstellung ist Stand der Technik: RepoGraph, KGCompass, CodexGraph und MCP-basierte Code-Knowledge-Graphen (Codebase-Memory, 2026) zeigen auf SWE-bench deutliche Verbesserungen. **Diese Arbeit beansprucht den Graphaufbau nicht als Beitrag**, sondern setzt darauf auf. Die Lücke liegt eine Ebene höher:

1. Alle genannten Systeme werden auf **eingefrorenen Benchmark-Snapshots** evaluiert, nicht im realen Entwicklungsalltag mit menschlichen Entwicklern.
2. Keines untersucht **Retrieval-Transparenz als Interaktionsschnittstelle**: die Möglichkeit, die Kontextauswahl vor der Ausführung zu inspizieren und zu korrigieren.

Abgrenzung zu existierenden Transparenz- und Interventionsansätzen. Einzelne Bausteine dieser Lücke sind besetzt, ihre Kombination ist es nicht:

- *Kontexttransparenz als Designeigenschaft*: AOCI (Liu et al. 2026) führt „Context transparency and editability“ explizit als Qualitätsdimension von Kontext-Repräsentationen ein, allerdings nur als Eigenschaft eines statischen Plain-Text-Index. Es gibt weder ein Inspektions-Interface noch eine Nutzerstudie; evaluiert wird rein computational.
- *Intervention in Nachbarmänen*: Memory Sandbox (Huang et al. 2023) und Memolet (Yen & Zhao 2024) machen Konversations-Memory einsehbar und editierbar; RAGViz (Wang et al. 2024) erlaubt das Zu-/Abschalten abgerufener Dokumente in RAG-QA. Cocoa (Feng et al. 2026) und WaitGPT (2024) lassen Nutzer auf Plan- bzw. Code-Ebene eingreifen. Keiner adressiert jedoch unmittelbar die aufgabenspezifische Kontextauswahl für Coding Agents; der Effekt von Nutzerkorrekturen auf das Aufgabenergebnis bleibt dort weitgehend offen.
- *Intervention in Produkten*: Sourcegraph Cody zeigt den intendierten Kontext vor dem Absenden als löschbare Context-Chips; GitHub Copilot Edits lässt Nutzer ein Working Set kuratieren und schlägt verwandte Dateien vor (VS Code v1.96, 2024). Beides sind flache Dateilisten **ohne Begründung der Auswahl**, und beide sind bislang kaum empirisch evaluiert.

Offen bleibt damit die Kombination aus (1) automatisch assembliertem, aufgabenspezifischem Kontext für Coding Agents, (2) begründeter Auswahl (Entscheidungspfad),

(3) Inspektion und Korrektur vor der Ausführung und (4) empirischer Messung von Fehlererkennung (HF2) und Ergebniswirkung (HF3). Insbesondere der Loop „Entwickler korrigiert Kontextauswahl → Agent-Ergebnis“ ist bislang kaum systematisch untersucht.

Der Graph ist dabei das *Mittel*, nicht der Zweck: Die Kontextauswahl entsteht aus einer **hybriden Pipeline**: semantische Embeddings und exakte Namens-/Pfad-Treffer liefern Seed-Knoten, die explizite Graphstruktur expandiert sie entlang nachvollziehbarer Kanten („`checkout()` aufgenommen via CALLS-Kante von `cart.ts`“). Entscheidend ist, dass *jeder* Schritt inspizierbar wird: Seed-Scores, Schwellwert und Near-Misses ebenso wie die Traversal-Kanten. Reines Embedding-Retrieval liefert dagegen nur eine opake Rangliste ohne Begründung. Das, nicht eine weitere Benchmark-Verbesserung, ist das Argument für den graphzentrierten, inspizierbaren Ansatz.

3 Forschungsfragen und Hypothesen

- **HF1 (Effizienz)**: Verbessert proaktiv graph-assemblierter Kontext (B4) die Effizienz von Coding Agents (Token-Verbrauch, Iterationsanzahl, Erstkorrektheit) gegenüber reaktiver Eigenexploration (B1), statischer Entwickler-Dokumentation (B2) und manuell kuratiertem Code-Kontext (B3)?

H1: Ja, gestützt auf ContextBench (Überexploration reaktiver Agents), Agentless (prozedurale Kontextvorbereitung schlägt Exploration) und die begrenzten Effekte statischer Kontext-Dateien.

- **HF2 (Transparenz)**: Ermöglicht ein expliziter Entscheidungspfad (Begründung jeder Kontextaufnahme über Graph-Kanten) Entwicklern, fehlerhafte Kontextauswahlen vor der Agentausführung zu erkennen?

H2: Entwickler mit Entscheidungspfad erkennen mehr eingebaute Kontextfehler als Entwickler, die nur die Kontextliste sehen.

- **HF3 (Intervention)**: Führt die Korrektur der Kontextauswahl durch Entwickler vor der Ausführung zu besseren Agent-Ergebnissen (Erstkorrektheit, weniger Korrekturiterationen)?

H3: Explorativ; der Loop „Entwickler editiert Kontext → Agent läuft“ ist bislang kaum systematisch untersucht.

4 Artefakt

Ein **MCP-Server**, der eine Codebase als Wissensgraphen (Neo4j mit Vektorindex, tree-sitter-Parsing für Python/TypeScript, lokale Embeddings via Ollama/`nomic-embed-text`)

repräsentiert und drei Funktionen bereitstellt:

1. **Context Assembly:** Für eine Aufgabenbeschreibung werden zunächst Seed-Knoten über eine **hybride Suche** (semantische Embedding-Ähnlichkeit *und* exakte Namens-/Pfad-Treffer) gefunden und anschließend per Graph-Traversal (strukturell, historisch über Git-Commits und GitHub-Issues/PRs, testbasiert) zu einem aufgabenspezifischen Kontext-Block expandiert, begrenzt auf ein Tokenbudget.
2. **Entscheidungspfad als First-Class-Output:** Jeder Kontext-Block enthält eine maschinen- und menschenlesbare Begründung pro Knoten: Ähnlichkeitsscore und überlappende Query-Tokens für Seeds bzw. die konkrete Graph-Kante und Traversal-Kette für expandierte Knoten (`decisionPath[]` inkl. Neo4j-Browser-Links).
3. **Inspektions- und Korrektur-Interface:** Vor der Agentenausführung öffnet sich ein browserbasiertes Review-UI (interaktiver Graph), das den geplanten Teilgraphen samt Retrieval-Trace zeigt. Der Entwickler kann jeden Knoten inspizieren (*warum* aufgenommen, mit Score bzw. Kante), Knoten entfernen oder ergänzen, die Suchanfrage editieren und neu abfeuern und den Retrieval-Vorgang Schritt für Schritt nachvollziehen (Kontext als editierbarer Vorschlag statt Black Box).

Die automatische Synchronisation per Git-Hook (inkrementelles Re-Parsing geänderter Dateien) ist notwendige Infrastruktur für den Einsatz an aktiven Projekten, wird aber als Implementierungsdetail behandelt; sie ist in existierenden Tools bereits gelöst.

Umsetzung und Stand (Juni 2026): Die vollständige Umsetzung des Artefakts ist expliziter, konstruktiver Bestandteil der Arbeit. Ein lauffähiger Prototyp belegt bereits die Machbarkeit: Parser (tree-sitter, Python/TypeScript), Neo4j-Graph mit Vektorindex, die fünfstufige hybride Context-Assembly-Pipeline, der global registrierte MCP-Server sowie ein erstes interaktives Review-Interface (`get_context_interactive`). Eine erste Embedding-Auswertung zeigt brauchbare Kontextabdeckung (Context Recall 2-Hop im Mittel 0,66 über fünf Aufgaben; ein Commit-Boost hebt Precision@3 von 0,07 auf 0,27). Im Verlauf der Arbeit werden Artefakt und Inspektions-Interface zur Studienreife ausgebaut (inkl. Living-Graph-Synchronisation und automatischem Session-Logging) und anschließend empirisch untersucht.

5 Methodik

Rahmen: Design Science Research (Hevner et al. 2004), iterative Build-Evaluate-Zyklen.

Evaluation in zwei Studien:

- **Studie 1 (HF1, Effizienz):** Vier Kontextbedingungen an realen Aufgaben: **B1** nur Aufgabenbeschreibung (reaktive Exploration), **B2** zusätzlich statische `CLAUDE.md`, **B3** zusätzlich manuell ausgewählte Code-Snippets, **B4** `CLAUDE.md` plus strukturierter Graph-Kontext. Aufgabentypen: isolierte Bugfixes (T1), Multi-File-Refactorings (T2), Feature-Entwicklung (T3). Metriken: Iterationsanzahl, Dauer, MCP-/Tool-Aufrufe und Erstkorrektheit; für die Graph-Bedingung automatisch über Claude-Code-Hooks als JSONL je Session geloggt und ausgewertet.
- **Studie 2 (HF2/HF3, Transparenz):** Kontrollierte Studie mit Studierenden, durchgeführt am implementierten Review-Interface (`get_context_interactive`). Aufgaben mit teilweise **gezielt verfälschten Kontextauswahlen** (seeded errors); gemessen wird die Erkennungsrate mit vs. ohne Entscheidungspfad (HF2) sowie der Effekt von Kontextkorrekturen (Knoten entfernen/ergänzen, Query editieren) auf das Agent-Ergebnis (HF3). Methodisch angelehnt an die AgentStepper-Studie, nur vor statt nach der Ausführung.

Versuchsumgebung: Aktiv entwickelte Hochschulprojekte (Python/TypeScript). Als Pilot- und Referenzumgebung dient ein realer Webshop (Next.js + Sanity + Shopify), perspektivisch ergänzt um weitere Projekte des Betreuers. Der Feldcharakter ist bewusst gewählt: Der Wert von Transparenz und projektspezifischem Kontext ist auf eingefrorenen SWE-bench-Snapshots prinzipiell nicht messbar.

6 Erwarteter Beitrag

1. **Empirisch:** Eine der ersten systematischen Untersuchungen von Retrieval-Transparenz und Pre-Execution-Kontextkorrektur bei Coding Agents; Effizienzvergleich vierer Kontext-Engineering-Strategien im Feldeinsatz statt auf Benchmarks.
2. **Konstruktiv:** Ein offenes, MCP-kompatibles Artefakt, das Kontextauswahl erklärbar und editierbar macht.
3. **Praktisch:** Handlungsempfehlungen, wann sich graphbasiertes Kontext-Engineering gegenüber statischen Kontext-Dateien lohnt.

Quellen zur Abgrenzung (Recherche-Stand: 11.06.2026)

- **AgentStepper:** Interactive Debugging of Software Development Agents. [arXiv:2602.06593](https://arxiv.org/abs/2602.06593) (2026). Trajektorien-Debugging mit Breakpoints und Live-Editing; Nutzerstudie n=12.

- **AOCI:** Liu et al.: Symbolic-Semantic Indexing for Practical Repository-Scale Code Understanding with LLMs. [arXiv:2605.02421](#) (2026). Benennt „Context transparency and editability“ als Dimension (Tab. 1), ohne Interface/Nutzerstudie.
- **Dhanorkar et al.:** Human Oversight of Agentic Systems in Practice. [arXiv:2606.05391](#) (2026). Interviewstudie n=17; identifiziert „a priori control“ als Oversight-Form ohne Werkzeugunterstützung.
- **Memory Sandbox:** Huang et al., [UIST '23 Adjunct](#) / [arXiv:2308.01542](#). Konversations-Memory ansehen, editieren, löschen; Design-Probe ohne summative Studie.
- **Memolet:** Yen & Zhao, [UIST '24](#). Konversations-Memories als interaktive, wiederverwendbare Objekte.
- **RAGViz:** Wang et al., EMNLP 2024 (Demo), [arXiv:2411.01751](#). Attention-Visualisierung + Dokument-Toggling in RAG-QA; Diagnose-Werkzeug ohne Ergebnisstudie.
- **Cocoa:** Feng et al., CHI 2026, [arXiv:2412.10999](#). Interaktive Pläne (Co-Planning/Co-Execution); Intervention auf Plan-, nicht Kontextebene.
- **WaitGPT:** [UIST '24](#), [arXiv:2408.01703](#). Monitoring/Steering von LLM-generiertem Analyse-Code während der Ausführung; n=12.
- **ContextBench:** A Benchmark for Context Retrieval in Coding Agents. [arXiv:2602.05892](#) (2026). Misst Überexploration reaktiver Agents; Pre-Submission-Kontextdeklaration durch den *Agenten*, nicht den Nutzer.
- **Sourcegraph Cody:** [Context-Dokumentation](#). Context-Chips vor dem Absenden löschar/ersetzbar; flache Dateiliste ohne Begründung, unevaluiert.
- **GitHub Copilot Edits:** [VS Code v1.96 Release Notes](#) (Dez. 2024). Working-Set-Kuration mit Related-Files-Vorschlägen (Git-Co-Change-Heuristik), unevaluiert.